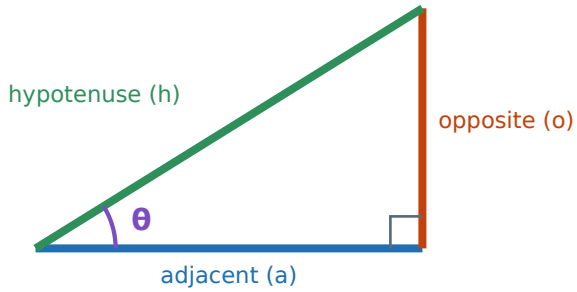
CREATIVE CODING · TU DUBLIN

Trig for Games: the Unit Circle in Godot

Right-angle triangles, the unit circle, and the handful of functions that move, aim and rotate almost everything in a 2D game. Theory first, then eight game patterns with GDScript, then problems with worked answers at the end.

1. The right-angle triangle



Any vector in 2D can be split into a horizontal part and a vertical part. Those two parts and the vector itself form a right-angle triangle.

The angle θ is measured at the corner where the **adjacent** side and the **hypotenuse** meet. The **opposite** side is across from it.

Three ratios relate the angle to the sides. SOH CAH TOA is the usual way to remember them:

```
sin θ = o / h      cos θ = a / h      tan θ = o / a
```

And Pythagoras gives the length of the hypotenuse from the other two sides:

```
h = √(a² + o²)
```

Going backwards, from sides to angle, uses the inverse functions. In code you almost always want `atan2(o, a)` rather than `atan(o / a)`, because atan2 looks at the signs of both values and returns the correct angle in all four quadrants (and doesn't divide by zero when a = 0).

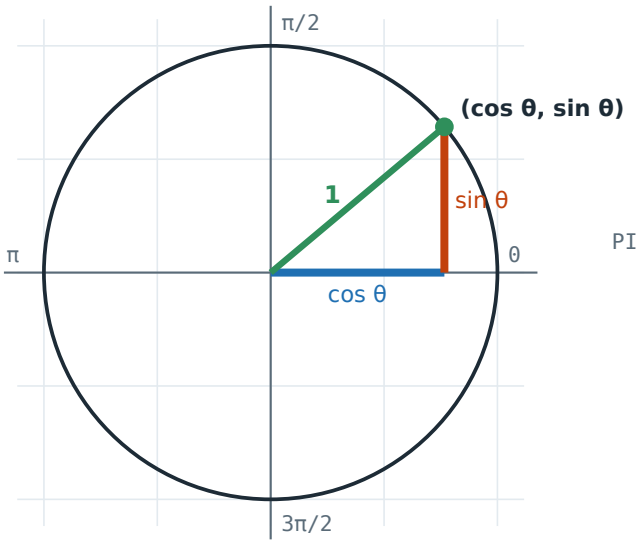
2. The unit circle

Fix h = 1. Then sin θ = o and cos θ = a, so the point at angle θ on a circle of radius 1 is simply:

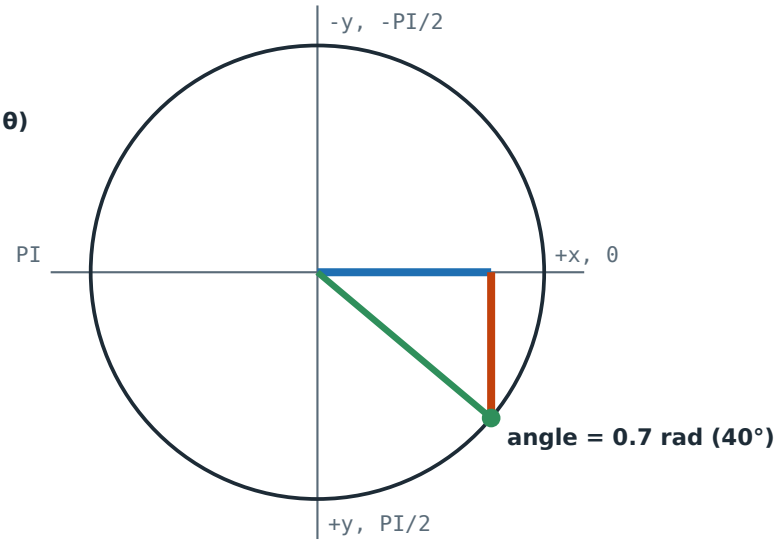
```
( cos θ, sin θ )
```

That is all the unit circle is: cos gives the x of a point on a radius-1 circle, sin gives its y. Both always lie between −1 and 1. For any other radius r, scale by r. For a circle centred somewhere other than the origin, add the centre:

```
x = cx + r·cos θ      y = cy + r·sin θ
```



maths convention: y up, angles anticlockwise



Godot 2D: y down, angles clockwise on screen

Godot flips y. In Godot 2D, +y points down the screen. The maths is identical, but positive angles turn clockwise on screen and positive sin moves things down. Rotation 0 points right (+x). To go up the screen you use a negative y, or the angle `-PI / 2`.

Radians. Godot's trig functions and `rotation` use radians. A full turn is TAU (2π ≈ 6.283); half a turn is PI. Convert with `deg_to_rad()` and `rad_to_deg()`, or use `rotation_degrees` on a node.

Values worth knowing

angle	radians	cos	sin	screen direction in Godot
0°	0	1	0	right
30°	π/6	0.866	0.5	right, slightly down
45°	π/4	0.707	0.707	diagonal down-right
60°	π/3	0.5	0.866	mostly down
90°	π/2	0	1	down
180°	π	−1	0	left
270° / −90°	3π/2 / −π/2	0	−1	up

3. The Godot toolkit

You can do everything with `sin`, `cos` and `atan2`, but Godot's `Vector2` wraps the common cases. Each line below is the same maths as section 2.

GDScript	What it does	Underlying trig
`Vector2.from_angle(a)`	Unit vector pointing at angle a	(cos a, sin a)
`v.angle()`	Angle of vector v	atan2(v.y, v.x)
`v.length()`	Length of v	$\sqrt{(x^2 + y^2)}$
`a.distance_to(b)`	Distance between two points	(b − a).length()
`a.angle_to_point(b)`	Angle from point a towards b	(b − a).angle()
`v.normalized()`	Same direction, length 1	v / v.length()
`v.rotated(a)`	Rotate v by angle a	rotation matrix of cos/sin
`node.look_at(p)`	Set rotation to face point p	rotation = angle_to_point(p)
`lerp_angle(a, b, t)`	Blend angles, taking the short way round	handles wrap at ±π
`angle_difference(a, b)`	Signed shortest angle from a to b	result in −π..π

4. Trig in games: eight patterns

4.1 Orbit

Moons, shield orbs, satellites, loading spinners. Advance an angle over time and place the node with x = cx + r·cos, y = cy + r·sin.

```
extends Node2D

@export var radius := 120.0
@export var speed := 1.5   # radians per second

var angle := 0.0

func _process(delta):
    angle += speed * delta
    $Moon.position = Vector2(cos(angle), sin(angle)) * radius
    # same thing: $Moon.position = Vector2.from_angle(angle) * radius
```

4.2 Move in the direction you're facing

Asteroids ships, top-down tanks, cars. Rotate with input, then turn the rotation into a direction vector.

```
extends CharacterBody2D

@export var turn_speed := 3.0
@export var thrust := 250.0

func _physics_process(delta):
    rotation += Input.get_axis("ui_left", "ui_right") * turn_speed * delta
    var forward = Vector2.from_angle(rotation)   # (cos r, sin r)
    velocity = forward * thrust * Input.get_action_strength("ui_up")
    move_and_slide()
```

4.3 Aim at a target

Turrets, twin-stick shooters, enemies that face the player. Subtract positions to get the vector to the target, then take its angle (atan2 under the hood). lerp_angle gives a smooth turn instead of a snap.

```
extends Node2D

@export var turn_rate := 5.0

func _process(delta):
    var to_mouse = get_global_mouse_position() - global_position
    var target_angle = to_mouse.angle()        # atan2(to_mouse.y, to_mouse.x)
    rotation = lerp_angle(rotation, target_angle, turn_rate * delta)
```

4.4 Fire a bullet

The bullet's velocity is the barrel's direction times its speed. Spawning it at the muzzle uses the same idea: start at the turret and step out along the direction.

```
const BULLET = preload("res://bullet.tscn")

func fire():
    var dir = Vector2.from_angle(global_rotation)
    var b = BULLET.instantiate()
    b.global_position = global_position + dir * 40   # 40 px out = muzzle
    b.velocity = dir * 600
    get_tree().current_scene.add_child(b)
```

4.5 Spread shots and bullet rings

Shotguns fire n bullets fanned across a spread angle. Bullet-hell rings fire n bullets spaced evenly around a full turn, TAU / n apart.

```
func fire_spread(n: int, spread_deg: float):
    var spread = deg_to_rad(spread_deg)
    for i in n:
        var t = 0.5 if n == 1 else float(i) / (n - 1)   # 0..1
        var a = global_rotation - spread / 2 + spread * t
        spawn_bullet(Vector2.from_angle(a))

func fire_ring(n: int):
    for i in n:
        spawn_bullet(Vector2.from_angle(i * TAU / n))
```

4.6 Bobbing, pulsing, swaying

sin(time) swings smoothly between −1 and 1, so it's ideal for anything that oscillates: floating pickups, breathing idle animations, pulsing UI, swaying grass. Multiply for size, multiply time for speed.

```
extends Sprite2D

@export var height := 8.0      # pixels up and down
@export var cycles := 0.5      # bobs per second

var t := 0.0
@onready var start_y := position.y

func _process(delta):
    t += delta
    position.y = start_y + sin(t * TAU * cycles) * height
    scale = Vector2.ONE * (1.0 + 0.05 * sin(t * TAU))   # gentle pulse
```

4.7 Range and vision cone

Distance is Pythagoras. A vision cone checks whether the angle between where the guard faces and where the player is falls inside half the cone's width.

```
@export var view_range := 300.0
@export var view_angle_deg := 60.0   # total cone width

func can_see(target: Vector2) -> bool:
    var to_target = target - global_position
    if to_target.length() > view_range:
        return false
    var diff = angle_difference(global_rotation, to_target.angle())
    return abs(diff) <= deg_to_rad(view_angle_deg) / 2
```

4.8 Launching a projectile

Angry Birds, grenades, artillery. Split the launch speed into horizontal and vertical components with cos and sin, then let gravity bend it. The minus sign makes the shot go up, because Godot's y points down.

```
extends CharacterBody2D

const GRAVITY = 980.0

func launch(speed: float, angle_deg: float):
    var a = deg_to_rad(angle_deg)
    velocity = Vector2(cos(a) * speed, -sin(a) * speed)

func _physics_process(delta):
    velocity.y += GRAVITY * delta
    move_and_slide()
```

5. Problems

Use a calculator where needed. Give angles in degrees and radians where asked. From problem 3 on, assume Godot 2D coordinates (y down). Answers are on the next page.

1. A right-angle triangle has adjacent side 3 and opposite side 4. Find the hypotenuse and the angle θ.

2. What are the coordinates of the point on the unit circle at 60°? At 150°?

3. The player is at (100, 100) and an enemy is at (300, 200). How far apart are they? What angle (in radians) should the enemy's turret face to aim at the player?

4. A moon orbits a planet at (400, 300) with radius 100. Where is the moon when its angle is 0? When it's PI / 2? When it's PI? Describe each position on screen.

5. A shotgun fires 5 pellets across a 60° spread, centred on rotation 0. List the angle of each pellet in degrees.

6. A cannonball is launched at 30° above horizontal with speed 400 px/s. What are velocity.x and velocity.y at launch in Godot?

7. A student writes `rotation = atan2(to_player.x, to_player.y)` and the turret points the wrong way. What's the bug? Give two correct versions.

8. Write one line of GDScript that makes a coin bob 6 px above and below its start height, completing 2 full bobs per second. Use variables t (seconds elapsed) and start_y.

9. A guard faces right (rotation 0) with a 90° vision cone and range 250. Can it see a player at offset (200, 150) from the guard? At (100, −120)?

10. Using only cos and sin, write a loop that places 12 hour markers around a clock face of radius 150 centred at (0, 0), with 12 o'clock at the top.

Answers

1. h = √(3² + 4²) = √25 = **5**. θ = atan2(4, 3) ≈ **53.13°** (0.927 rad).

2. 60°: (cos 60°, sin 60°) = **(0.5, 0.866)**. 150°: (−0.866, 0.5).

3. Vector from enemy to player = (100, 100) − (300, 200) = (−200, −100). Distance = √(200² + 100²) = √50000 ≈ **223.6 px**. Angle = atan2(−100, −200) ≈ **−2.678 rad** (−153.4°): left and slightly up on screen. In code: `(player - enemy).angle()` or `enemy.angle_to_point(player)`.

4. 0 → (500, 300), right of the planet. PI / 2 → (400, 400), **below** the planet, because +y is down. PI → (300, 300), left of the planet.

5. Step = 60 / (5 − 1) = 15°. Angles: **−30°, −15°, 0°, 15°, 30°**.

6. velocity.x = 400 · cos 30° ≈ **346.4**. velocity.y = −400 · sin 30° = **−200** (negative so it rises).

7. The arguments are swapped: atan2 takes y first, then x. Correct: `rotation = atan2(to_player.y, to_player.x)` or `rotation = to_player.angle()`. (`look_at(player.global_position)` also works.)

8. `position.y = start_y + sin(t * TAU * 2) * 6`

9. (200, 150): distance = 250, inside range (just). Angle = atan2(150, 200) ≈ 36.9°, within ±45°. **Yes**. (100, −120): distance ≈ 156.2, in range. Angle = atan2(−120, 100) ≈ −50.2°, outside ±45°. **No**.

10. 12 o'clock is up, which is −PI / 2 in Godot. Each hour is TAU / 12 apart, going clockwise on screen:

```
for i in 12:
    var a = -PI / 2 + i * TAU / 12
    var pos = Vector2(cos(a), sin(a)) * 150
    draw_circle(pos, 4, Color.WHITE)   # inside _draw()
```

Going further: try combining patterns. A turret that aims at the player (4.3), only when the player is inside its vision cone (4.7), and fires a spread (4.5), uses nothing beyond what's on these pages.